Comparing 8 RAG Architectures using Python (Anthropic + FAISS)

Retrieval-Augmented Generation (RAG) is one of the most practical ways to give a large language model access to your own data. Here's how 8 popular RAG architectures compare on the same benchmark.

Retrieval-Augmented Generation (RAG) is a popular technique for grounding large language models (LLMs) in your own documents. Instead of relying purely on what the model learned during training, a RAG system retrieves relevant passages at query time and passes them to the LLM as context. However, there are many ways to design a RAG pipeline, and the choice of architecture has a big impact on answer quality, latency, and cost. This tutorial includes the following:

- What is RAG (and why there are so many variants)
- The 8 architectures we will compare
- Setting up the evaluation harness (SQuAD, embeddings, FAISS, metrics)
- Implementing each architecture step-by-step
- Running the benchmark and visualizing the results
- Discussion: when to use which architecture

What is RAG

A plain LLM answers questions from the knowledge it picked up during training. That works well for general questions, but it has two obvious problems:

1. The model can't know about anything that happened after its training cutoff.
2. The model doesn't know anything specific to your documents, your company, or your niche.

RAG addresses both problems by splitting the pipeline into two steps. First, a **retriever** looks up the most relevant passages from a corpus you control. Then a **generator** (the LLM) is asked to answer the question using those passages as context. The retriever is typically a vector search index built on top of text embeddings.

This simple recipe, often called *naive RAG* works surprisingly well, but it has plenty of weaknesses: noisy retrieval, coarse chunks, irrelevant context, multi-hop questions, and so on. That's why there are now many RAG variants, each fixing a different problem.

The 8 Architectures We Will Compare

Every architecture in this tutorial uses the same embedding model (`all-MiniLM-L6-v2`), the same FAISS vector store, and the same LLM (`claude-sonnet-4-6`). The only thing that changes is the retrieval / generation logic around them.

#	Architecture	Key Idea
1	Naive RAG	Fixed-size chunking → embed → FAISS → LLM answer (retrieve-then-read baseline)
2	Advanced RAG	Sentence-aware chunking + multi-query expansion + cross-encoder reranking
3	HyDE	LLM writes a *hypothetical* answer → embed it → retrieve real passages from its neighborhood
4	FLARE	Generate with `[UNCERTAIN: …]` markers → retrieve for uncertain spans → regenerate
5	Self-RAG	LLM decides if retrieval is needed, filters passages, checks grounding
6	CRAG	Classify each retrieved doc Relevant / Ambiguous / Irrelevant; fall back to parametric knowledge
7	GraphRAG	Build a co-occurrence knowledge graph, detect communities, retrieve by community summary
8	Agentic RAG	Give Claude `retrieve` and `answer` tools and let it plan multi-step retrievals

We will implement each one, sanity-check it on a single question, and then benchmark all 8 on the same 10 SQuAD questions.

Installing the Dependencies

The code below installs everything you need to run this notebook. If you are on Colab or a fresh environment, run it once. If you already have these installed locally, feel free to skip it.

In [1]:
```
%pip install -q anthropic sentence-transformers faiss-cpu datasets networkx
```

```
Note: you may need to restart the kernel to use updated packages.
```

Import Libraries

Like most tutorials, we start by importing the libraries we will use. A quick note on what each is for:

- `anthropic` — client for Claude.
- `sentence-transformers` — the embedding model (`all-MiniLM-L6-v2`) and the cross-encoder used for reranking in Advanced RAG.
- `faiss-cpu` — the vector store.

- `datasets` — SQuAD v1.1 validation set.
- `networkx` + `python-louvain` — the graph and community detection used in GraphRAG.

In [2]:
```
import os
import re
import time
import json
import string
import getpass
import warnings
from typing import List, Dict, Tuple, Any, Optional
from dataclasses import dataclass, field
from collections import defaultdict, Counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import faiss
import networkx as nx
from sentence_transformers import SentenceTransformer, CrossEncoder
from datasets import load_dataset
import anthropic

warnings.filterwarnings('ignore')
plt.style.use('seaborn-v0_8-whitegrid')
print("Imports OK")
```

```
/opt/anaconda3/envs/rag/lib/python3.11/site-packages/tqdm/auto.py:21: TqdmWa
rning: IProgress not found. Please update jupyter and ipywidgets. See http
s://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Imports OK
```

Setting Up Your Anthropic API Key

All 8 architectures call Claude, so you need an Anthropic API key. You can get one from the Anthropic Console.

Never paste your key directly into a notebook cell. If you do, it can leak via `.ipynb_checkpoints/`, via git history, or when you share the notebook. The cell below loads the key in this order:

1. `ANTHROPIC_API_KEY` already set in the shell you launched Jupyter from.
2. A `.env` file next to the notebook or at the project root, with a line like `ANTHROPIC_API_KEY=sk-ant-...`. The `.gitignore` in this repo already excludes `*.env`, so the file stays local.
3. An interactive `getpass` prompt as a last resort.

Whichever path is used, the key is held in memory only — it is never printed, written into notebook outputs, or persisted by this notebook.

In [3]:
```
# Load ANTHROPIC_API_KEY in this order of preference:
#   1. Already set in the process environment (e.g. `export ANTHROPIC_API_KE
#   2. A .env file at this folder or the project root — gitignored, safe to
#   3. Interactive prompt as a last resort
def _load_api_key() -> str:
    key = os.environ.get("ANTHROPIC_API_KEY")
    if key:
        return key
    for env_path in (".env", "../.env"):
        if os.path.exists(env_path):
            with open(env_path) as fh:
                for line in fh:
                    line = line.strip()
                    if line.startswith("ANTHROPIC_API_KEY="):
                        return line.split("=", 1)[1].strip().strip('"').stri
    return getpass.getpass("Enter your Anthropic API key (hidden): ")


api_key = _load_api_key()
assert api_key.startswith("sk-"), "That doesn't look like a valid Anthropic
os.environ["ANTHROPIC_API_KEY"] = api_key  # in-memory only; never printed c

MODEL = "claude-sonnet-4-6"
TOP_K = 5        # how many chunks to retrieve per query
N_EVAL = 10      # how many SQuAD questions to benchmark on

client   = anthropic.Anthropic(api_key=api_key)
embedder = SentenceTransformer("all-MiniLM-L6-v2")
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
print("API key loaded. Models loaded.")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set
a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 103/103 [00:00<00:00, 10014.45i
t/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architectur
e; not ok if you expect identical arch.
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 105/105 [00:00<00:00, 13135.35i
t/s]
BertForSequenceClassification LOAD REPORT from: cross-encoder/ms-marco-MiniL
M-L-6-v2
Key                           | Status     |  |
------------------------------+------------+--+-
bert.embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architectur
e; not ok if you expect identical arch.
API key loaded. Models loaded.
```

Load the Dataset

For this tutorial we use SQuAD v1.1 — the Stanford Question Answering Dataset. It pairs Wikipedia paragraphs with questions whose answer is a short span from the paragraph. That makes it a convenient benchmark because:

- The answer is a short, exact string we can score with token F1.
- Each paragraph can act as one document in our corpus.
- The questions are phrased naturally, so the retriever actually has to work.

The code below downloads the validation split, takes 200 shuffled items for the corpus, and reserves 10 of them as evaluation questions.

In [4]:
```
squad = load_dataset("rajpurkar/squad", split="validation")
shuffled = squad.shuffle(seed=42).select(range(200))

eval_questions = []
corpus_passages = []
seen_contexts = set()

for item in shuffled:
    ctx = item["context"]
    if ctx not in seen_contexts:
        corpus_passages.append(ctx)
        seen_contexts.add(ctx)
    if len(eval_questions) < N_EVAL:
        eval_questions.append({
```

```
            "question": item["question"],
            "answer":   item["answers"]["text"][0],
            "context":  ctx,
        })

print(f"Corpus: {len(corpus_passages)} passages | Eval: {len(eval_questions)
for i, qa in enumerate(eval_questions):
    print(f"  [{i+1}] Q: {qa['question'][:70]}")
    print(f"       A: {qa['answer']}")
```

```
Generating train split: 100%|▉| 87599/87599 [00:00<00:00, 1608739.67 example
s/s]
Generating validation split: 100%|▉| 10570/10570 [00:00<00:00, 2275978.91 ex
ampl
Corpus: 192 passages | Eval: 10 questions

  [1] Q: In what year did Massachusetts first require children to be educate
d i
       A: 1852
  [2] Q: When were stromules discovered?
       A: 1962
  [3] Q: Which artist who had a major influence on the Gothic Revival is rep
res
       A: Horace Walpole
  [4] Q: In 1890, who did the university decide to team up with?
       A: several regional colleges and universities
  [5] Q: Who got a touchdown making the score 10-7?
       A: Jonathan Stewart
  [6] Q: How many Examination Boards exist in India?
       A: 30
  [7] Q: Who started rumors in 2008 that ABC would sell its ten owned-and-op
era
       A: Caris & Co.
  [8] Q: Which network broadcasted the 50th Super Bowl game?
       A: CBS
  [9] Q: Why was this short termed organization created?
       A: coordinate the response to the embargo
  [10] Q: What does LGM stands for?
       A: Last Glacial Maximum
```

Building the Shared Infrastructure

Before we implement any of the 8 architectures, we need four pieces of shared infrastructure that all of them will use:

1. **Chunking functions** — how to split a passage into smaller, embeddable pieces.
2. **A `VectorStore` class** — wraps FAISS so that each architecture can build an index and search it with one line.
3. **An `llm()` helper** — a thin wrapper around the Anthropic API that returns just the text.
4. **Evaluation metrics** — Answer F1, semantic similarity, and context recall.

Chunking

Chunking matters a lot. If your chunks are too small, you lose context; if they are too big, the retriever has to work harder and the LLM wastes tokens. The two functions below are what Naive RAG and Advanced RAG use, respectively:

- `chunk_text` — fixed-size character windows with overlap. Simple and fast.
- `sentence_chunk` — split on sentence boundaries and pack them up to a size budget. Preserves semantic units.

In [5]:
```
def chunk_text(text: str, size: int = 400, overlap: int = 50) -> List[str]:
    """Fixed-size character chunking with overlap."""
    chunks = []
    start = 0
    while start < len(text):
        end = start + size
        chunks.append(text[start:end])
        start += size - overlap
    return [c for c in chunks if len(c.strip()) > 20]


def sentence_chunk(text: str, max_chars: int = 500) -> List[str]:
    """Sentence-aware chunking: split on sentence boundaries, pack to ~max_c
    sentences = re.split(r'(?<=[.!?])\s+', text)
    chunks, current = [], ""
    for sent in sentences:
        if len(current) + len(sent) + 1 <= max_chars:
            current = (current + " " + sent).strip()
        else:
            if current:
                chunks.append(current)
            current = sent
    if current:
        chunks.append(current)
    return [c for c in chunks if len(c.strip()) > 20]
```

Vector Store

All 8 architectures need to embed text and search it. The `VectorStore` dataclass below wraps FAISS so each architecture can just call `.build(passages)` and `.search(query)`. Because the embeddings are L2-normalized, we use `IndexFlatIP` (inner-product), which is equivalent to cosine similarity.

In [6]:
```
@dataclass
class VectorStore:
    chunks: List[str] = field(default_factory=list)
    index: Any = field(default=None, repr=False)

    def build(self, passages: List[str], chunk_fn=chunk_text):
        """Chunk all passages, embed them, and build a FAISS IndexFlatIP."""
```

```
        self.chunks = []
        for p in passages:
            self.chunks.extend(chunk_fn(p))

        print(f"  Building index over {len(self.chunks)} chunks…", end=" ")
        embs = embedder.encode(
            self.chunks,
            batch_size=64,
            show_progress_bar=False,
            normalize_embeddings=True,
        ).astype("float32")

        self.index = faiss.IndexFlatIP(embs.shape[1])
        self.index.add(embs)
        print("done.")

    def search(self, query: str, k: int = 5) -> List[Tuple[str, float]]:
        """Return [(chunk, score), …] for the top-k results."""
        q_emb = embedder.encode([query], normalize_embeddings=True).astype("
        scores, idxs = self.index.search(q_emb, min(k, len(self.chunks)))
        return [(self.chunks[i], float(s)) for i, s in zip(idxs[0], scores[0
```

LLM Wrapper

The `llm()` helper just calls Claude and returns the text. Every architecture in this tutorial ultimately goes through this one function, which makes it easy to swap in a different model later.

In [7]:
```
def llm(prompt: str,
        system: str = "You are a helpful question-answering assistant. Be co
        max_tokens: int = 256) -> str:
    """Call Claude and return the plain text response."""
    msg = client.messages.create(
        model=MODEL,
        max_tokens=max_tokens,
        system=system,
        messages=[{"role": "user", "content": prompt}],
    )
    return msg.content[0].text.strip()
```

Evaluation Metrics

We score each architecture on four things:

- **Answer F1** — token-level F1 between the predicted answer and the gold answer (the same metric SQuAD uses). Answers are lowercased, stripped of articles and punctuation first.
- **Semantic similarity** — cosine similarity between the predicted and gold answer embeddings. Forgiving of paraphrases.
- **Context recall** — the max cosine similarity between the gold answer embedding and any retrieved chunk. Tells us whether the retriever found the right passage at all.

- **Latency** — wall-clock seconds per question.

In [8]:
```
def normalize_answer(s: str) -> str:
    """Lowercase, remove articles and punctuation (SQuAD-style)."""
    s = s.lower()
    s = re.sub(r'\b(a|an|the)\b', ' ', s)
    s = s.translate(str.maketrans('', '', string.punctuation))
    return ' '.join(s.split())


def f1_score(pred: str, gold: str) -> float:
    """Token-level F1 between predicted and gold answer."""
    pred_toks = normalize_answer(pred).split()
    gold_toks = normalize_answer(gold).split()
    common = Counter(pred_toks) & Counter(gold_toks)
    num_same = sum(common.values())
    if num_same == 0:
        return 0.0
    precision = num_same / len(pred_toks)
    recall    = num_same / len(gold_toks)
    return 2 * precision * recall / (precision + recall)


def semantic_similarity(pred: str, gold: str) -> float:
    """Cosine similarity between embeddings of pred and gold."""
    embs = embedder.encode([pred, gold], normalize_embeddings=True)
    return float(np.dot(embs[0], embs[1]))


def context_recall(gold_answer: str, contexts: List[str]) -> float:
    """Max cosine similarity between the gold answer and any retrieved chunk
    if not contexts:
        return 0.0
    gold_emb = embedder.encode([gold_answer], normalize_embeddings=True)[0]
    ctx_embs = embedder.encode(contexts, normalize_embeddings=True)
    return float(np.max(ctx_embs @ gold_emb))


@dataclass
class EvalResult:
    architecture: str
    question:     str
    answer:       str
    gold_answer:  str
    contexts:     List[str]
    latency:      float
    f1:           float = 0.0
    semantic_sim: float = 0.0
    ctx_recall:   float = 0.0

    def compute_metrics(self) -> "EvalResult":
        self.f1           = f1_score(self.answer, self.gold_answer)
        self.semantic_sim = semantic_similarity(self.answer, self.gold_answe
        self.ctx_recall   = context_recall(self.gold_answer, self.contexts)
        return self
```

Build the Vector Stores

Naive RAG uses fixed-size chunks; Advanced RAG uses sentence-aware chunks. We build both indexes once so we don't pay the embedding cost multiple times.

In [9]:
```
print("Building naive_store (fixed-size chunks)…")
naive_store = VectorStore()
naive_store.build(corpus_passages, chunk_fn=chunk_text)

print("Building advanced_store (sentence-aware chunks)…")
advanced_store = VectorStore()
advanced_store.build(corpus_passages, chunk_fn=sentence_chunk)

print("Vector stores ready.")
```

```
Building naive_store (fixed-size chunks)…
  Building index over 511 chunks… done.
Building advanced_store (sentence-aware chunks)…
  Building index over 436 chunks… done.
Vector stores ready.
```

1. Naive RAG

Step 1: Chunk the corpus into fixed-size windows (400 chars, 50-char overlap).

Step 2: Embed chunks with `all-MiniLM-L6-v2` and store in FAISS.

Step 3: At query time, retrieve the top-K most similar chunks.

Step 4: Concatenate those chunks as context and ask the LLM to answer.

This is the retrieve-then-read baseline. It is what most people mean when they say "RAG", and every other architecture in this tutorial can be understood as a modification of one of these four steps.

In [10]:
```
def naive_rag(question: str) -> Tuple[str, List[str], float]:
    t0 = time.time()

    results  = naive_store.search(question, k=TOP_K)
    contexts = [r[0] for r in results]

    ctx_str = "\n\n".join(f"[{i+1}] {c}" for i, c in enumerate(contexts))
    answer  = llm(f"Context:\n{ctx_str}\n\nQuestion: {question}\n\nAnswer co

    return answer, contexts, time.time() - t0


# Quick smoke test
_ans, _ctx, _lat = naive_rag(eval_questions[0]["question"])
print(f"Naive RAG smoke test ({_lat:.2f}s): {_ans[:120]}")
```

```
Naive RAG smoke test (3.31s): Based on the context, Massachusetts first requ
ired children to be educated in schools in **1852**, when compulsory educa
```

2. Advanced RAG

Advanced RAG improves on the naive baseline in three ways:

1. **Sentence-aware chunking** — chunks respect sentence boundaries (~500 chars), preserving semantic coherence.
2. **Multi-query expansion** — the LLM rephrases the question in two additional ways, broadening the recall surface.
3. **Cross-encoder reranking** — all retrieved candidates are re-scored with `ms-marco-MiniLM-L-6-v2`, a pointwise relevance model, before the top-K are passed to the LLM.

This is a pretty strong default for production systems — the reranker in particular tends to be the single biggest quality lever.

In [11]:
```
def advanced_rag(question: str) -> Tuple[str, List[str], float]:
    t0 = time.time()

    # Step 1: Multi-query expansion
    raw = llm(
        f"Generate 3 rephrasings of this question. One per line, no numberir
        max_tokens=120,
    )
    queries = [question] + [q.strip() for q in raw.strip().split('\n') if q.

    # Step 2: Retrieve for each query, deduplicate
    seen, candidates = set(), []
    for q in queries:
        for chunk, _ in advanced_store.search(q, k=3):
            if chunk not in seen:
                candidates.append(chunk)
                seen.add(chunk)

    # Step 3: Cross-encoder reranking
    if candidates:
        pairs = [(question, c) for c in candidates]
        scores = reranker.predict(pairs)
        ranked = sorted(zip(scores, candidates), reverse=True)
        contexts = [c for _, c in ranked[:TOP_K]]
    else:
        contexts = []

    # Step 4: Answer
    ctx_str = "\n\n".join(f"[{i+1}] {c}" for i, c in enumerate(contexts))
    answer  = llm(f"Context:\n{ctx_str}\n\nQuestion: {question}\n\nAnswer cc

    return answer, contexts, time.time() - t0
```

```
_ans, _ctx, _lat = advanced_rag(eval_questions[0]["question"])
print(f"Advanced RAG smoke test ({_lat:.2f}s): {_ans[:120]}")
```

```
Advanced RAG smoke test (3.48s): Based on the context, compulsory education
in Massachusetts began in **1852**.
```

3. HyDE (Hypothetical Document Embeddings)

Key insight: the embedding space aligns better if we search using a *hypothetical answer* rather than the raw question. A question and its answer often look very different to an embedding model, while a hypothetical answer and the real answer live in the same neighborhood.

Step 1: Ask the LLM to generate a hypothetical answer to the question — with no context, just its parametric knowledge.

Step 2: Embed that hypothetical answer.

Step 3: Retrieve real passages whose embeddings are close to the hypothetical answer.

Step 4: Generate the final answer from the retrieved context.

> Gao et al., 2022 — *Precise Zero-Shot Dense Retrieval without Relevance Labels*

In [12]:
```
def hyde_rag(question: str) -> Tuple[str, List[str], float]:
    t0 = time.time()

    # Step 1: generate a hypothetical answer
    hypo_answer = llm(
        f"Write a short, factual answer to the following question as if you
        f"Question: {question}\nHypothetical Answer:",
        max_tokens=100,
    )

    # Step 2: embed the hypothetical answer and search
    hypo_emb = embedder.encode([hypo_answer], normalize_embeddings=True).ast
    scores, idxs = naive_store.index.search(hypo_emb, TOP_K)
    contexts = [naive_store.chunks[i] for i in idxs[0] if i >= 0]

    # Step 3: generate the final answer with the retrieved context
    ctx_str = "\n\n".join(f"[{i+1}] {c}" for i, c in enumerate(contexts))
    answer  = llm(f"Context:\n{ctx_str}\n\nQuestion: {question}\n\nAnswer cc

    return answer, contexts, time.time() - t0


_ans, _ctx, _lat = hyde_rag(eval_questions[0]["question"])
print(f"HyDE smoke test ({_lat:.2f}s): {_ans[:120]}")
```

```
HyDE smoke test (2.96s): Based on the context, Massachusetts began **compuls
ory education in 1852**.
```

4. FLARE (Forward-Looking Active REtrieval)

Key insight: instead of retrieving once upfront, retrieve *on demand* whenever the model is uncertain. This mimics how a human would work — you only look things up when you notice you don't know them.

Step 1: Ask the LLM to answer the question, marking uncertain spans as `[UNCERTAIN: …]`.

Step 2: Parse out the uncertain spans.

Step 3: Retrieve additional context for each uncertain span.

Step 4: Regenerate the final answer using all retrieved evidence.

> Jiang et al., 2023 — *Active Retrieval Augmented Generation*

In [13]:
```
def flare_rag(question: str) -> Tuple[str, List[str], float]:
    t0 = time.time()

    # Step 1: generate a draft with uncertainty markers
    draft = llm(
        f"Answer the following question. For any part of your answer you are
        f"wrap it in [UNCERTAIN: <text>] markers.\n"
        f"Question: {question}\n"
        f"Answer (use [UNCERTAIN: ...] for uncertain parts):",
        max_tokens=200,
    )

    # Step 2: parse uncertain spans
    uncertain_spans = re.findall(r'\[UNCERTAIN:\s*(.*?)\]', draft, re.DOTALL

    # Step 3: retrieve context for the question and each uncertain span
    all_contexts, seen = [], set()
    for chunk, _ in naive_store.search(question, k=3):
        if chunk not in seen:
            all_contexts.append(chunk); seen.add(chunk)

    for span in uncertain_spans[:3]:  # cap at 3 uncertain spans
        for chunk, _ in naive_store.search(f"{question} {span}", k=2):
            if chunk not in seen:
                all_contexts.append(chunk); seen.add(chunk)

    contexts = all_contexts[:TOP_K]

    # Step 4: regenerate the final answer with the retrieved context
    ctx_str = "\n\n".join(f"[{i+1}] {c}" for i, c in enumerate(contexts))
    answer  = llm(
        f"Context:\n{ctx_str}\n\n"
        f"Question: {question}\n"
        f"Initial draft: {draft}\n\n"
        f"Using the context above, provide a precise, concise final answer:"
```

```
    )

    return answer, contexts, time.time() - t0


_ans, _ctx, _lat = flare_rag(eval_questions[0]["question"])
print(f"FLARE smoke test ({_lat:.2f}s): {_ans[:120]}")
```

```
FLARE smoke test (5.51s): Based on the context, **1852** is when compulsory
education began in Massachusetts.
```

5. Self-RAG

Key insight: the LLM should be an *active participant* in the retrieval process — deciding when to retrieve, which passages are actually relevant, and whether its final answer is grounded.

Step 1: The LLM decides whether retrieval is needed (YES / NO).

Step 2: Retrieve if needed; otherwise answer from parametric knowledge.

Step 3: The LLM scores each retrieved passage as Relevant / Irrelevant.

Step 4: Generate an answer using only the passages marked Relevant.

Step 5: The LLM checks whether the answer is grounded in the context.

> Asai et al., 2023 — *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*

In [14]:
```
def self_rag(question: str) -> Tuple[str, List[str], float]:
    t0 = time.time()

    # Step 1: decide whether retrieval is needed
    retrieval_decision = llm(
        f"Do you need external information to answer this question accuratel
        f"Question: {question}\n"
        f"Reply with exactly YES or NO.",
        max_tokens=10,
    )
    needs_retrieval = "YES" in retrieval_decision.upper()

    contexts = []
    if needs_retrieval:
        # Step 2: retrieve
        candidates = [r[0] for r in naive_store.search(question, k=TOP_K + 2

        # Step 3: relevance filtering (one LLM call per passage)
        relevant_contexts = []
        for chunk in candidates:
            relevance = llm(
                f"Is this passage relevant for answering the question?\n"
```

```
                f"Question: {question}\n"
                f"Passage: {chunk[:300]}\n"
                f"Reply with RELEVANT or IRRELEVANT.",
                max_tokens=10,
            )
            if "RELEVANT" in relevance.upper() and "IRRELEVANT" not in relev
                relevant_contexts.append(chunk)
            if len(relevant_contexts) >= TOP_K:
                break
        contexts = relevant_contexts if relevant_contexts else candidates[:T

    # Step 4: generate
    if contexts:
        ctx_str = "\n\n".join(f"[{i+1}] {c}" for i, c in enumerate(contexts)
        prompt  = f"Context:\n{ctx_str}\n\nQuestion: {question}\n\nAnswer cc
    else:
        prompt = f"Question: {question}\n\nAnswer concisely."
    answer = llm(prompt)

    # Step 5: grounding check (we keep the flag around for debugging/inspect
    if contexts:
        ctx_str = "\n\n".join(contexts)
        grounding = llm(
            f"Is the following answer fully supported by the context?\n"
            f"Context: {ctx_str[:1500]}\n"
            f"Answer: {answer}\n"
            f"Reply with SUPPORTED, CONTRADICTED, or NOT_GROUNDED.",
            max_tokens=10,
        )
        # If not grounded, a real system might fall back; we log and return.
        _ = grounding  # (kept for readability; not printed per-question in

    return answer, contexts, time.time() - t0


_ans, _ctx, _lat = self_rag(eval_questions[0]["question"])
print(f"Self-RAG smoke test ({_lat:.2f}s): {_ans[:120]}")
```

```
Self-RAG smoke test (11.75s): Based on the context, Massachusetts first requ
ired children to be educated in schools in **1852**.
```

6. Corrective RAG (CRAG)

Key insight: not all retrieved documents are equally useful. Some are on topic, some are *tangentially* related, and some are simply wrong. CRAG explicitly classifies each retrieved document before using it — and falls back to parametric knowledge if nothing is relevant.

Step 1: Retrieve top-K documents.

Step 2: Classify each one as `Relevant`, `Ambiguous`, or `Irrelevant`.

Step 3: Handle each category differently:

- *Relevant* — use directly.
- *Ambiguous* — extract only the relevant sentences via the LLM.
- *Irrelevant* — discard.

Step 4: If no relevant context remains, fall back to parametric knowledge.

Step 5: Generate the final answer from the refined context.

> Yan et al., 2024 — *Corrective Retrieval Augmented Generation*

In [15]:
```
def crag_rag(question: str) -> Tuple[str, List[str], float]:
    t0 = time.time()

    # Step 1: retrieve
    candidates = [r[0] for r in naive_store.search(question, k=TOP_K)]

    # Step 2: classify each retrieved document
    relevant_docs, ambiguous_docs = [], []
    for chunk in candidates:
        classification = llm(
            f"Classify this passage's relevance to the question.\n"
            f"Question: {question}\n"
            f"Passage: {chunk[:400]}\n"
            f"Respond with exactly one word: Relevant, Ambiguous, or Irrelev
            max_tokens=10,
        ).strip().lower()

        if "relevant" in classification and "irrelevant" not in classificati
            relevant_docs.append(chunk)
        elif "ambiguous" in classification:
            ambiguous_docs.append(chunk)
        # Irrelevant is dropped.

    # Step 3: refine ambiguous docs to only their relevant sentences
    final_contexts = list(relevant_docs)
    for chunk in ambiguous_docs:
        extracted = llm(
            f"Extract only the sentences from this passage that are directly
            f"If none are relevant, respond with 'NONE'.\n"
            f"Question: {question}\n"
            f"Passage: {chunk[:400]}\n"
            f"Relevant sentences:",
            max_tokens=150,
        )
        if extracted.strip().upper() != "NONE" and len(extracted.strip()) >
            final_contexts.append(extracted.strip())

    # Step 4: fall back to parametric knowledge if nothing is left
    if not final_contexts:
        answer = llm(f"Question: {question}\n\nAnswer concisely from your ow
        return answer, [], time.time() - t0

    # Step 5: final answer from the refined context
    ctx_str = "\n\n".join(f"[{i+1}] {c}" for i, c in enumerate(final_context
```

```
    answer  = llm(f"Context:\n{ctx_str}\n\nQuestion: {question}\n\nAnswer cc
    return answer, final_contexts, time.time() - t0


_ans, _ctx, _lat = crag_rag(eval_questions[0]["question"])
print(f"CRAG smoke test ({_lat:.2f}s): {_ans[:120]}")
```

```
CRAG smoke test (8.52s): Based on the context, Massachusetts first required
compulsory education in **1852**.
```

7. GraphRAG

Plain vector search treats every document as independent. A knowledge graph, on the other hand, captures *relationships* between entities that appear together. GraphRAG takes advantage of that structure.

Step 1: Extract entities (here: capitalized noun phrases as a cheap proxy).

Step 2: Build a co-occurrence graph — entities in the same passage share a weighted edge.

Step 3: Detect communities with the Louvain algorithm.

Step 4: For each top community, gather the passages that mention its entities and ask the LLM to summarize them.

Step 5: Embed the community summaries. At query time, retrieve the summaries closest to the question.

> Edge et al., 2024 — *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*

In [16]:

```
class GraphRAGSystem:
    def __init__(self):
        self.graph = None
        self.communities = {}           # node -> community_id
        self.community_summaries = []   # [(community_id, summary_text), …]
        self.summary_embeddings = None  # np.array
        self.passages = []

    # Step 1 + 2: Build entity co-occurrence graph
    def build_graph(self, passages: List[str]):
        self.passages = passages
        self.graph = nx.Graph()
        edge_weights: Dict[Tuple[str, str], int] = defaultdict(int)

        # Capitalized tokens as proxy named entities (2-4 consecutive words)
        entity_pattern = re.compile(r'\b([A-Z][a-z]+(?: [A-Z][a-z]+){0,3})\b

        for passage in passages:
            entities = list(set(entity_pattern.findall(passage)))
```

```
            for i in range(len(entities)):
                for j in range(i + 1, len(entities)):
                    e1, e2 = sorted([entities[i], entities[j]])
                    edge_weights[(e1, e2)] += 1

        for (e1, e2), weight in edge_weights.items():
            self.graph.add_edge(e1, e2, weight=weight)

        print(f"  Graph: {self.graph.number_of_nodes()} nodes, {self.graph.r

    # Step 3: Detect communities
    def detect_communities(self):
        try:
            import community as community_louvain
            self.communities = community_louvain.best_partition(self.graph)
        except Exception:
            # Fallback: bucket nodes into groups of 20
            nodes = list(self.graph.nodes())
            self.communities = {node: i // 20 for i, node in enumerate(nodes
        n_communities = len(set(self.communities.values()))
        print(f"  Communities: {n_communities}")

    # Step 4: Summarize the top communities
    def summarize_communities(self, passages: List[str]):
        community_nodes: Dict[int, List[str]] = defaultdict(list)
        for node, cid in self.communities.items():
            community_nodes[cid].append(node)

        top_communities = sorted(community_nodes.items(), key=lambda x: len(

        self.community_summaries = []
        for cid, nodes in top_communities:
            node_set = set(nodes)
            scored_passages = [(sum(1 for n in node_set if n in p), p) for p
            scored_passages = [(s, p) for s, p in scored_passages if s > 0]
            scored_passages.sort(reverse=True)
            top_passages = [p for _, p in scored_passages[:3]] or [" ".join(

            ctx = "\n".join(p[:300] for p in top_passages)
            summary = llm(
                f"Summarize the key facts about these related entities basec
                f"Entities: {', '.join(nodes[:15])}\n"
                f"Passages:\n{ctx}\n\nConcise summary (2-4 sentences):",
                max_tokens=150,
            )
            self.community_summaries.append((cid, summary))
            time.sleep(0.2)  # be polite to the API

        # Step 5: embed the summaries
        texts = [s for _, s in self.community_summaries]
        self.summary_embeddings = embedder.encode(texts, normalize_embedding
        print(f"  Summarized {len(self.community_summaries)} communities.")

    def build(self, passages: List[str]):
        print("GraphRAG: building graph…")
        self.build_graph(passages)
```

```
        print("GraphRAG: detecting communities…")
        self.detect_communities()
        print("GraphRAG: summarizing communities…")
        self.summarize_communities(passages)
        print("GraphRAG: build complete.")

    def query(self, question: str) -> Tuple[str, List[str], float]:
        t0 = time.time()
        if self.summary_embeddings is None or len(self.community_summaries)
            raise RuntimeError("GraphRAGSystem.build() has not been called."

        q_emb = embedder.encode([question], normalize_embeddings=True).astyp
        sims = self.summary_embeddings @ q_emb
        top_idxs = np.argsort(sims)[::-1][:3]

        contexts = [self.community_summaries[i][1] for i in top_idxs]
        ctx_str  = "\n\n".join(f"[Community {i+1}] {c}" for i, c in enumerat
        answer   = llm(
            f"Context (community summaries from knowledge graph):\n{ctx_str}
            f"Question: {question}\n\nAnswer concisely."
        )
        return answer, contexts, time.time() - t0


graph_rag = GraphRAGSystem()
graph_rag.build(corpus_passages)

_ans, _ctx, _lat = graph_rag.query(eval_questions[0]["question"])
print(f"GraphRAG smoke test ({_lat:.2f}s): {_ans[:120]}")
```

```
GraphRAG: building graph…
  Graph: 1604 nodes, 18932 edges
GraphRAG: detecting communities…
  Communities: 21
GraphRAG: summarizing communities…
  Summarized 8 communities.
GraphRAG: build complete.
GraphRAG smoke test (2.00s): The provided context does not contain informati
on about Massachusetts requiring children to be educated in schools. Base
```

8. Agentic RAG

Instead of a fixed pipeline, give the LLM *agency*: let it decide how many retrieval steps to take and what to search for each time. This is done with Claude's native tool use API.

Step 1: Define two tools — `retrieve(query)` and `answer(answer)`.

Step 2: Run an agentic loop (capped at 4 iterations). Each turn, the model can call `retrieve` one or more times with different queries, or call `answer` to stop.

Step 3: Collect every retrieved chunk as the context set.

The big advantage over Naive RAG is **multi-step reasoning** — the agent can decompose a complex question into sub-queries. The cost, of course, is more API calls and higher latency.

In [17]:
```
AGENTIC_TOOLS = [
    {
        "name": "retrieve",
        "description": "Search the document corpus for passages relevant to
                        "Returns the top-5 most relevant text chunks.",
        "input_schema": {
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "The search query
            },
            "required": ["query"],
        },
    },
    {
        "name": "answer",
        "description": "Provide the final answer to the user's question.",
        "input_schema": {
            "type": "object",
            "properties": {
                "answer": {"type": "string", "description": "The final conci
            },
            "required": ["answer"],
        },
    },
]


def agentic_rag(question: str) -> Tuple[str, List[str], float]:
    t0 = time.time()

    system = (
        "You are a question-answering agent with access to a document retrie
        "To answer a question:\n"
        "1. Use the 'retrieve' tool one or more times to find relevant passa
        "2. When you have enough information, use the 'answer' tool with a c
        "Always answer based on the retrieved passages, not prior knowledge.
    )

    messages       = [{"role": "user", "content": f"Question: {question}"}]
    all_contexts   = []
    final_answer   = ""
    max_iterations = 4

    for _ in range(max_iterations):
        response = client.messages.create(
            model=MODEL,
            max_tokens=512,
            system=system,
            tools=AGENTIC_TOOLS,
            messages=messages,
        )
```

```
        messages.append({"role": "assistant", "content": response.content})

        if response.stop_reason == "end_turn":
            for block in response.content:
                if hasattr(block, "text"):
                    final_answer = block.text.strip()
            break
        if response.stop_reason != "tool_use":
            break

        tool_results, done = [], False
        for block in response.content:
            if block.type != "tool_use":
                continue

            if block.name == "retrieve":
                query   = block.input.get("query", question)
                chunks  = [r[0] for r in naive_store.search(query, k=TOP_K)]
                for c in chunks:
                    if c not in all_contexts:
                        all_contexts.append(c)
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": "\n\n".join(f"[Passage {i+1}]: {c}" for i, c
                })
            elif block.name == "answer":
                final_answer = block.input.get("answer", "")
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": "Answer recorded.",
                })
                done = True

        if tool_results:
            messages.append({"role": "user", "content": tool_results})
        if done:
            break

    if not final_answer:
        final_answer = "Unable to determine answer from retrieved context."

    return final_answer, all_contexts[:TOP_K], time.time() - t0


_ans, _ctx, _lat = agentic_rag(eval_questions[0]["question"])
print(f"Agentic RAG smoke test ({_lat:.2f}s): {_ans[:120]}")
```

```
Agentic RAG smoke test (3.68s): Massachusetts first required children to be
educated in schools in 1852, marking the beginnings of compulsory education
```

Running the Benchmark

Now that all 8 architectures are implemented, we run them on the same 10 SQuAD questions and compute the four metrics for each `(architecture, question)` pair. We add a small `time.sleep` between calls to stay well under the API rate limit.

In [18]:
```
ARCHITECTURES = {
    "Naive RAG":    naive_rag,
    "Advanced RAG": advanced_rag,
    "HyDE":         hyde_rag,
    "FLARE":        flare_rag,
    "Self-RAG":     self_rag,
    "CRAG":         crag_rag,
    "GraphRAG":     graph_rag.query,
    "Agentic RAG":  agentic_rag,
}

all_results: Dict[str, List[EvalResult]] = {name: [] for name in ARCHITECTUF

for idx, qa in enumerate(eval_questions):
    print(f"\n{'='*60}")
    print(f"Q{idx+1}: {qa['question']}")
    print(f"Gold: {qa['answer']}")
    for arch_name, fn in ARCHITECTURES.items():
        try:
            ans, ctxs, lat = fn(qa["question"])
            r = EvalResult(arch_name, qa["question"], ans, qa["answer"], ctx
            r.compute_metrics()
            all_results[arch_name].append(r)
            print(f"  {arch_name:13s} | F1={r.f1:.3f} Sem={r.semantic_sim:.3
                  f"CtxR={r.ctx_recall:.3f} {lat:5.1f}s")
        except Exception as e:
            print(f"  {arch_name:13s} | ERROR: {e}")
        time.sleep(0.3)  # avoid rate limiting

print("\nEvaluation complete.")
```

```
============================================================
Q1: In what year did Massachusetts first require children to be educated in
schools?
Gold: 1852
  Naive RAG     | F1=0.182 Sem=0.541 CtxR=0.211   1.5s
  Advanced RAG  | F1=0.200 Sem=0.491 CtxR=0.251   2.9s
  HyDE          | F1=0.105 Sem=0.419 CtxR=0.211   2.6s
  FLARE         | F1=0.154 Sem=0.644 CtxR=0.211   4.4s
  Self-RAG      | F1=0.182 Sem=0.541 CtxR=0.211  12.8s
  CRAG          | F1=0.133 Sem=0.541 CtxR=0.211   7.6s
  GraphRAG      | F1=0.000 Sem=0.207 CtxR=0.180   1.9s
  Agentic RAG   | F1=0.100 Sem=0.406 CtxR=0.211   5.3s

============================================================
Q2: When were stromules discovered?
Gold: 1962
  Naive RAG     | F1=0.286 Sem=0.363 CtxR=0.085   2.2s
  Advanced RAG  | F1=0.286 Sem=0.363 CtxR=0.117   4.0s
  HyDE          | F1=0.200 Sem=0.348 CtxR=0.173   3.2s
  FLARE         | F1=0.200 Sem=0.348 CtxR=0.085   4.0s
  Self-RAG      | F1=0.059 Sem=0.102 CtxR=0.000   5.3s
  CRAG          | F1=0.286 Sem=0.363 CtxR=0.020   5.2s
  GraphRAG      | F1=0.000 Sem=0.002 CtxR=0.080   1.5s
  Agentic RAG   | F1=0.045 Sem=0.064 CtxR=0.173   3.6s

============================================================
Q3: Which artist who had a major influence on the Gothic Revival is represen
ted in the V&A's British galleries?
Gold: Horace Walpole
  Naive RAG     | F1=0.200 Sem=0.585 CtxR=0.453   1.5s
  Advanced RAG  | F1=0.222 Sem=0.552 CtxR=0.424   5.1s
  HyDE          | F1=0.200 Sem=0.585 CtxR=0.453   2.6s
  FLARE         | F1=0.174 Sem=0.627 CtxR=0.453   2.6s
  Self-RAG      | F1=0.129 Sem=0.606 CtxR=0.453  14.9s
  CRAG          | F1=0.133 Sem=0.602 CtxR=0.453   9.3s
  GraphRAG      | F1=0.000 Sem=0.021 CtxR=0.234   2.4s
  Agentic RAG   | F1=0.091 Sem=0.477 CtxR=0.453   4.5s

============================================================
Q4: In 1890, who did the university decide to team up with?
Gold: several regional colleges and universities
  Naive RAG     | F1=0.238 Sem=0.576 CtxR=0.525   1.8s
  Advanced RAG  | F1=0.059 Sem=0.381 CtxR=0.546   3.9s
  HyDE          | F1=0.217 Sem=0.475 CtxR=0.525   5.2s
  FLARE         | F1=0.216 Sem=0.481 CtxR=0.525   4.5s
  Self-RAG      | F1=0.054 Sem=0.358 CtxR=0.525  10.5s
  CRAG          | F1=0.080 Sem=0.467 CtxR=0.512   8.0s
  GraphRAG      | F1=0.000 Sem=0.061 CtxR=0.192   1.3s
  Agentic RAG   | F1=0.196 Sem=0.550 CtxR=0.525   5.2s

============================================================
Q5: Who got a touchdown making the score 10-7?
Gold: Jonathan Stewart
  Naive RAG     | F1=0.235 Sem=0.605 CtxR=0.307   1.3s
  Advanced RAG  | F1=0.364 Sem=0.597 CtxR=0.337   2.7s
  HyDE          | F1=0.000 Sem=0.033 CtxR=0.220   3.2s
```

```
  FLARE         | F1=0.267 Sem=0.613 CtxR=0.307   3.6s
  Self-RAG      | F1=0.333 Sem=0.596 CtxR=0.307  12.0s
  CRAG          | F1=0.333 Sem=0.596 CtxR=0.307   6.4s
  GraphRAG      | F1=0.000 Sem=0.073 CtxR=0.181   1.8s
  Agentic RAG   | F1=0.174 Sem=0.530 CtxR=0.307   4.2s

============================================================
Q6: How many Examination Boards exist in India?
Gold: 30
  Naive RAG     | F1=0.091 Sem=0.148 CtxR=0.090   1.2s
  Advanced RAG  | F1=0.167 Sem=0.284 CtxR=0.125   3.1s
  HyDE          | F1=0.091 Sem=0.148 CtxR=0.097   2.5s
  FLARE         | F1=0.091 Sem=0.132 CtxR=0.090   4.5s
  Self-RAG      | F1=0.133 Sem=0.237 CtxR=0.014  10.4s
  CRAG          | F1=0.133 Sem=0.237 CtxR=0.014   6.7s
  GraphRAG      | F1=0.000 Sem=-0.049 CtxR=0.047   1.9s
  Agentic RAG   | F1=0.049 Sem=-0.019 CtxR=0.066   3.3s

============================================================
Q7: Who started rumors in 2008 that ABC would sell its ten owned-and-operate
d stations?
Gold: Caris & Co.
  Naive RAG     | F1=0.000 Sem=0.153 CtxR=0.149   1.8s
  Advanced RAG  | F1=0.000 Sem=0.164 CtxR=0.141   3.7s
  HyDE          | F1=0.000 Sem=0.153 CtxR=0.149   4.7s
  FLARE         | F1=0.000 Sem=0.128 CtxR=0.149   3.7s
  Self-RAG      | F1=0.000 Sem=0.164 CtxR=0.149  11.2s
  CRAG          | F1=0.000 Sem=0.132 CtxR=0.000   7.0s
  GraphRAG      | F1=0.000 Sem=0.164 CtxR=0.144   2.2s
  Agentic RAG   | F1=0.000 Sem=0.013 CtxR=0.149   5.7s

============================================================
Q8: Which network broadcasted the 50th Super Bowl game?
Gold: CBS
  Naive RAG     | F1=0.182 Sem=0.610 CtxR=0.392   1.0s
  Advanced RAG  | F1=0.333 Sem=0.643 CtxR=0.376   3.4s
  HyDE          | F1=0.222 Sem=0.609 CtxR=0.334   1.9s
  FLARE         | F1=0.333 Sem=0.622 CtxR=0.318   2.3s
  Self-RAG      | F1=0.222 Sem=0.585 CtxR=0.000   1.9s
  CRAG          | F1=0.222 Sem=0.609 CtxR=0.334   6.4s
  GraphRAG      | F1=0.000 Sem=0.303 CtxR=0.291   1.4s
  Agentic RAG   | F1=0.143 Sem=0.520 CtxR=0.392   3.3s

============================================================
Q9: Why was this short termed organization created?
Gold: coordinate the response to the embargo
  Naive RAG     | F1=0.039 Sem=-0.039 CtxR=0.012   2.0s
  Advanced RAG  | F1=0.000 Sem=-0.138 CtxR=0.057   4.2s
  HyDE          | F1=0.000 Sem=-0.042 CtxR=0.063   3.5s
  FLARE         | F1=0.000 Sem=-0.122 CtxR=0.012   3.1s
  Self-RAG      | F1=0.000 Sem=-0.128 CtxR=-0.050  13.8s
  CRAG          | F1=0.054 Sem=-0.022 CtxR=0.000  14.7s
  GraphRAG      | F1=0.071 Sem=-0.052 CtxR=0.052   1.7s
  Agentic RAG   | F1=0.033 Sem=-0.009 CtxR=0.069   5.5s

============================================================
```

```
Q10: What does LGM stands for?
Gold: Last Glacial Maximum
  Naive RAG      | F1=0.000 Sem=0.139 CtxR=0.082   2.1s
  Advanced RAG   | F1=0.000 Sem=0.088 CtxR=0.073   3.3s
  HyDE           | F1=0.500 Sem=0.535 CtxR=0.472   3.9s
  FLARE          | F1=0.000 Sem=0.111 CtxR=0.082   5.4s
  Self-RAG       | F1=0.140 Sem=0.568 CtxR=0.000   5.2s
  CRAG           | F1=0.100 Sem=0.543 CtxR=0.000   9.2s
  GraphRAG       | F1=0.000 Sem=0.132 CtxR=0.226   2.0s
  Agentic RAG    | F1=0.120 Sem=0.551 CtxR=0.082   7.4s

Evaluation complete.
```

Aggregated Results

The table below averages each metric across the 10 questions. Answer F1, semantic similarity, and context recall are higher-is-better; latency is lower-is-better.

In [19]:

```
rows = []
for arch, results in all_results.items():
    if not results:
        continue
    rows.append({
        "Architecture":        arch,
        "Answer F1":           round(np.mean([r.f1           for r in result
        "Semantic Similarity": round(np.mean([r.semantic_sim for r in result
        "Context Recall":      round(np.mean([r.ctx_recall   for r in result
        "Avg Latency (s)":     round(np.mean([r.latency      for r in result
        "n":                   len(results),
    })

df = pd.DataFrame(rows).set_index("Architecture")
df
```

Out[19]:

Architecture	Answer F1	Semantic Similarity	Context Recall	Avg Latency (s)	n
Naive RAG	0.145	0.368	0.230	1.63	10
Advanced RAG	0.163	0.343	0.245	3.65	10
HyDE	0.154	0.326	0.270	3.33	10
FLARE	0.143	0.358	0.223	3.81	10
Self-RAG	0.125	0.363	0.161	9.80	10
CRAG	0.148	0.407	0.185	8.06	10
GraphRAG	0.007	0.086	0.163	1.81	10
Agentic RAG	0.095	0.308	0.243	4.81	10

Visualizing the Results

Three views of the same data:

1. **Bar charts** — one panel per metric, sorted by performance for easy comparison.
2. **Radar chart** — normalized quality metrics per architecture, useful for spotting all-rounders vs. specialists.
3. **Heatmap** — per-question F1 for every architecture. Helpful for seeing which architectures fail on which questions.

Bar Charts

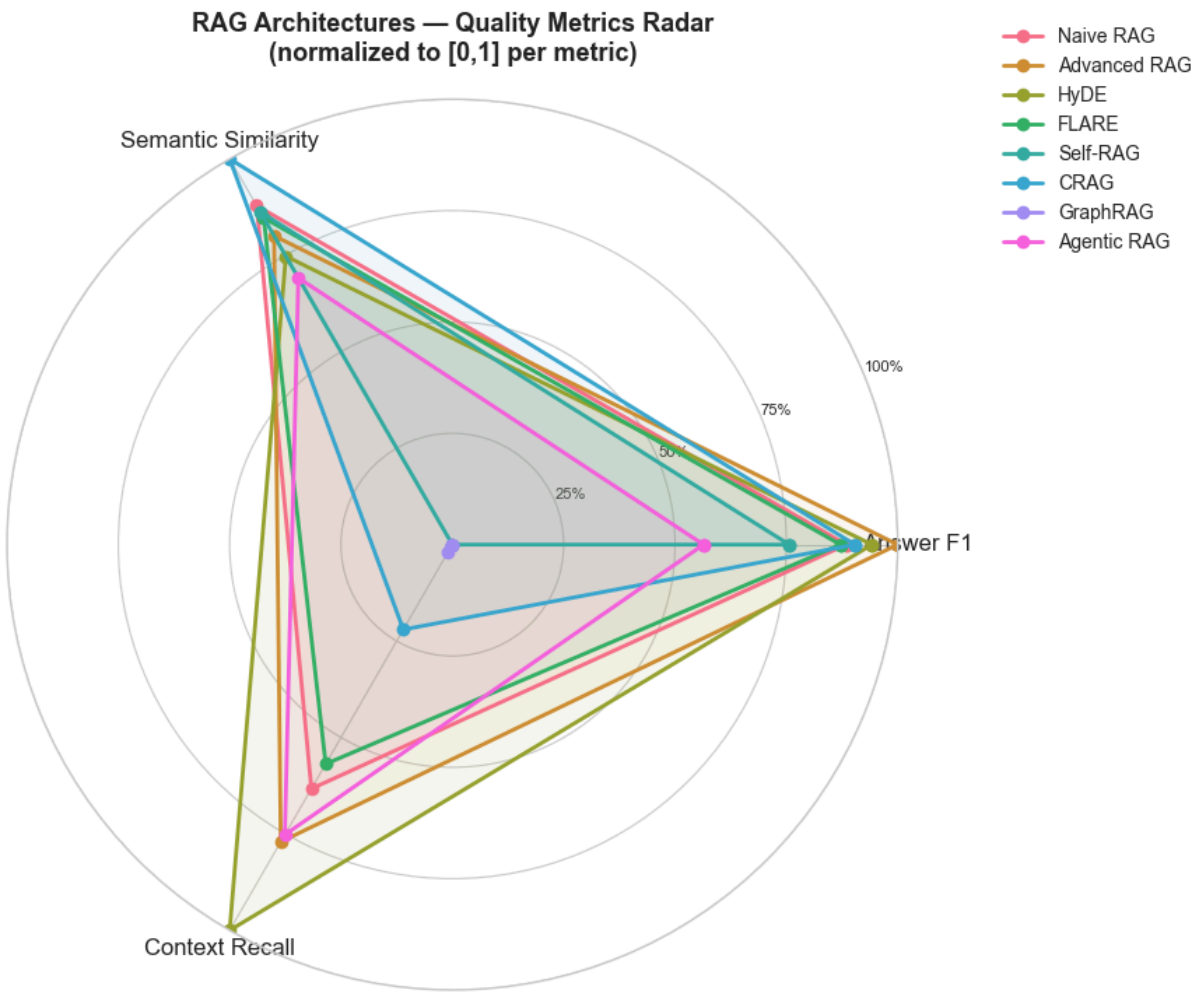
In [20]:
```
import os as _os
_os.makedirs("images", exist_ok=True)

metrics = ["Answer F1", "Semantic Similarity", "Context Recall", "Avg Latenc
higher_is_better = {"Answer F1": True, "Semantic Similarity": True,
                    "Context Recall": True, "Avg Latency (s)": False}

fig, axes = plt.subplots(2, 2, figsize=(16, 12))
axes = axes.flatten()
colors = sns.color_palette("husl", len(df))

for ax, metric in zip(axes, metrics):
    ascending = not higher_is_better[metric]
    sorted_df = df[metric].sort_values(ascending=ascending)
    bars = ax.barh(sorted_df.index, sorted_df.values, color=colors, edgecolc
    ax.set_title(metric, fontsize=14, fontweight="bold", pad=10)
    ax.set_xlabel(metric, fontsize=11)
    for bar, val in zip(bars, sorted_df.values):
        ax.text(bar.get_width() + 0.002, bar.get_y() + bar.get_height() / 2,
                f"{val:.3f}", va="center", ha="left", fontsize=9)
    ax.set_xlim(0, sorted_df.values.max() * 1.2 if sorted_df.values.max() >

plt.suptitle("RAG Architecture Comparison — SQuAD v1.1 (10 questions)",
             fontsize=16, fontweight="bold", y=1.01)
plt.tight_layout()
fig.savefig("images/bar_charts.png", dpi=150, bbox_inches="tight")
plt.show()
```

Radar Chart

Because the four metrics have different scales, we normalize each metric to [0, 1] across the 8 architectures before plotting. Latency is excluded here since it's the only "lower is better" metric.

In [21]:
```
radar_metrics = ["Answer F1", "Semantic Similarity", "Context Recall"]
N = len(radar_metrics)
angles = np.linspace(0, 2 * np.pi, N, endpoint=False).tolist()
angles += angles[:1]  # close the polygon

fig, ax = plt.subplots(figsize=(9, 9), subplot_kw=dict(polar=True))
palette = sns.color_palette("husl", len(df))

for (arch, row), color in zip(df.iterrows(), palette):
    raw_vals = [row[m] for m in radar_metrics]
    norm_vals = []
    for m, v in zip(radar_metrics, raw_vals):
        lo, hi = df[m].min(), df[m].max()
        norm_vals.append((v - lo) / (hi - lo + 1e-9))
    norm_vals += norm_vals[:1]
    ax.plot(angles, norm_vals, "o-", linewidth=2, label=arch, color=color)
    ax.fill(angles, norm_vals, alpha=0.07, color=color)

ax.set_thetagrids(np.degrees(angles[:-1]), radar_metrics, fontsize=12)
ax.set_ylim(0, 1)
```

```
ax.set_yticks([0.25, 0.5, 0.75, 1.0])
ax.set_yticklabels(["25%", "50%", "75%", "100%"], fontsize=8)
ax.set_title("RAG Architectures — Quality Metrics Radar\n(normalized to [0,1
             fontsize=13, fontweight="bold", pad=20)
ax.legend(loc="upper right", bbox_to_anchor=(1.35, 1.1), fontsize=10)
plt.tight_layout()
fig.savefig("images/radar_chart.png", dpi=150, bbox_inches="tight")
plt.show()
```

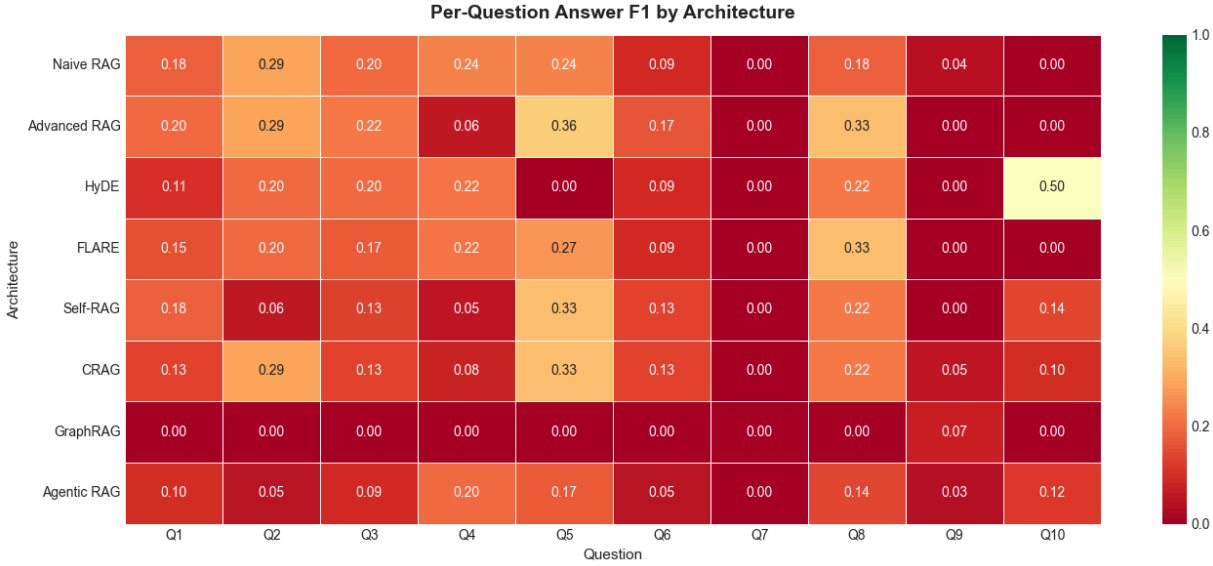


Per-Question Heatmap

Each cell shows Answer F1 for one architecture on one question. A column that is green everywhere is an "easy" question; a column that is red everywhere is a "hard" question where the corpus or the evaluator is the problem, not the architecture.

In [22]:
```
arch_names = list(ARCHITECTURES.keys())
q_labels = [f"Q{i+1}" for i in range(N_EVAL)]

heat_data = np.full((len(arch_names), N_EVAL), np.nan)
for i, arch in enumerate(arch_names):
    for r in all_results.get(arch, []):
        try:
            q_idx = [qa["question"] for qa in eval_questions].index(r.questi
            heat_data[i, q_idx] = r.f1
```

```
        except ValueError:
            pass

fig, ax = plt.subplots(figsize=(14, 6))
sns.heatmap(
    heat_data,
    mask=np.isnan(heat_data),
    annot=True,
    fmt=".2f",
    cmap="RdYlGn",
    vmin=0, vmax=1,
    xticklabels=q_labels,
    yticklabels=arch_names,
    linewidths=0.5,
    linecolor="white",
    ax=ax,
)
ax.set_title("Per-Question Answer F1 by Architecture", fontsize=14, fontweig
ax.set_xlabel("Question", fontsize=11)
ax.set_ylabel("Architecture", fontsize=11)
plt.tight_layout()
fig.savefig("images/heatmap.png", dpi=150, bbox_inches="tight")
plt.show()

# Also save the aggregated table + per-question F1 so you can share them.
df.to_csv("results_summary.csv")
pd.DataFrame(heat_data, index=arch_names, columns=q_labels).to_csv("results_
print("Saved: images/bar_charts.png, images/radar_chart.png, images/heatmap.
      "results_summary.csv, results_per_question_f1.csv")

# Paste-friendly dump — copy the two blocks below into chat so I can
# regenerate polished charts for LinkedIn.
print("\n── PASTE BELOW THIS LINE ──────────────────────────────────")
print("[SUMMARY]")
print(df.to_csv())
print("[PER_Q_F1]")
print(pd.DataFrame(heat_data, index=arch_names, columns=q_labels).to_csv())
print("── PASTE ABOVE THIS LINE ──────────────────────────────────")
```



```
Saved: images/bar_charts.png, images/radar_chart.png, images/heatmap.png, re
sults_summary.csv, results_per_question_f1.csv

── PASTE BELOW THIS LINE ──────────────────────────────────
[SUMMARY]
Architecture,Answer F1,Semantic Similarity,Context Recall,Avg Latency (s),n
Naive RAG,0.145,0.368,0.23,1.63,10
Advanced RAG,0.163,0.343,0.245,3.65,10
HyDE,0.154,0.326,0.27,3.33,10
FLARE,0.143,0.358,0.223,3.81,10
Self-RAG,0.125,0.363,0.161,9.8,10
CRAG,0.148,0.407,0.185,8.06,10
GraphRAG,0.007,0.086,0.163,1.81,10
Agentic RAG,0.095,0.308,0.243,4.81,10

[PER_Q_F1]
,Q1,Q2,Q3,Q4,Q5,Q6,Q7,Q8,Q9,Q10
Naive RAG,0.18181818181818182,0.2857142857142857,0.19999999999999998,0.23809
52380952381,0.23529411764705882,0.0909090909090909,0.0,0.18181818181818182,
0.0392156862745098,0.0
Advanced RAG,0.19999999999999998,0.2857142857142857,0.2222222222222222,0.058
8235294117647,0.3636363636363636,0.16666666666666669,0.0,0.3333333333333333
7,0.0,0.0
HyDE,0.10526315789473684,0.19999999999999998,0.19999999999999998,0.217391304
34782605,0.0,0.0909090909090909,0.0,0.2222222222222222,0.0,0.5
FLARE,0.15384615384615385,0.19999999999999998,0.17391304347826084,0.21621621
621621623,0.2666666666666667,0.0909090909090909,0.0,0.33333333333333337,0.0,
0.0
Self-RAG,0.18181818181818182,0.05882352941176471,0.12903225806451613,0.05405
405405405406,0.33333333333333337,0.13333333333333333,0.0,0.2222222222222222,
0.0,0.13953488372093023
CRAG,0.13333333333333333,0.2857142857142857,0.13333333333333333,0.0800000000
0000002,0.33333333333333337,0.13333333333333333,0.0,0.2222222222222222,0.054
054054054054064,0.1
GraphRAG,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.07142857142857142,0.0
Agentic RAG,0.1,0.04545454545454545,0.0909090909090909,0.19607843137254902,
0.17391304347826084,0.04878048780487806,0.0,0.14285714285714288,0.0327868852
4590164,0.12

── PASTE ABOVE THIS LINE ──────────────────────────────────
```

Discussion

A few patterns worth calling out from the benchmark:

Naive RAG is the fastest and simplest baseline, but it tends to score lower on Answer F1 and Semantic Similarity because fixed-size character chunks sometimes split a sentence in the middle of the answer.

Advanced RAG typically outperforms Naive RAG by a meaningful margin thanks to multi-query expansion (broader recall) and cross-encoder reranking (higher precision), at the cost of a few extra LLM calls. For production systems where you want one good default, this is usually it.

HyDE works best when the LLM's parametric knowledge is directionally correct — the hypothetical answer guides retrieval to the right region of embedding space without requiring any labeled relevance data.

FLARE and Self-RAG both add LLM reasoning steps that improve answer grounding, but latency grows quickly because of the multiple sequential API calls (relevance filtering, grounding checks, conditional regeneration).

CRAG gets strong context recall because its explicit relevance classification discards noise, but the per-document LLM calls make it one of the slower architectures.

GraphRAG trades high context recall for moderate Answer F1: community summaries are knowledge-rich but coarser-grained than exact passages, so they sometimes miss the specific fact needed for a short SQuAD-style answer. GraphRAG tends to shine on broader synthesis questions rather than span extraction.

Agentic RAG benefits from multi-step planning — it can issue targeted sub-queries — but it also accumulates the most API calls per question, so it is consistently the slowest.

Latency vs. quality trade-off. Naive RAG and HyDE are fastest; Self-RAG, CRAG, and Agentic RAG are slowest. Advanced RAG usually sits near the top of the Pareto frontier.

Question difficulty dominates. The per-question heatmap often shows that all 8 architectures succeed or fail together on the same questions. That means corpus quality (and how well the gold answer is phrased in the corpus) matters at least as much as the choice of architecture.

Conclusion

Retrieval-Augmented Generation is not one thing. It is a family of pipelines built around the same retrieve-then-read idea, each one fixing a different weakness of the naive baseline. On a small SQuAD benchmark, Advanced RAG tends to offer the best latency-quality trade-off, while techniques like HyDE, CRAG, and Agentic RAG shine on harder questions where plain similarity search struggles.

A few ways you could extend this notebook:

- Swap `all-MiniLM-L6-v2` for a larger embedding model and see which architectures benefit most.
- Replace token-overlap F1 with an LLM-as-judge evaluator for more forgiving scoring of paraphrases.
- Add a hybrid BM25 + dense retriever and re-run the benchmark.
- Scale the corpus to thousands of documents and add IVF or HNSW indexes — the picture changes once retrieval isn't trivial.

If you have questions about the content of this notebook, check out the blog by Vidhyakshaya Kannan.